

Algorithms for the Multidimensional Knapsack Problem (MKP)

Presented By

Shovon Roy – 2005010

Tusher Bhomik – 2005046

Md. Rabby Hossain – 2005053

Souvik Mandol – 2005069

Sagor Chanda – 2005071

Iffat Bin Hossain – 2005087

April 5, 2026

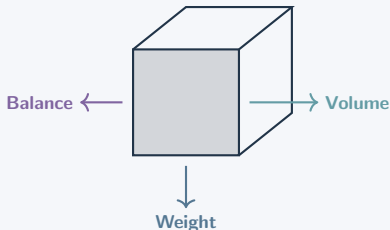
Problem Definition

Problem Definition: The "Cargo Plane" Analogy

The Intuition

Imagine you are loading a cargo plane. Unlike a standard knapsack (where you only care about weight), here you have **multiple limits** that must ALL be satisfied simultaneously:

- **Limit 1 (W):** Max Takeoff Weight.
- **Limit 2 (V):** Cargo Bay Volume.
- **Limit 3 (B):** Center of Gravity / Balance.



Formal Math

Maximize Profit $Z = \sum p_j x_j$ subject to:

$$\sum_{j=1}^n w_{ij} x_j \leq C_i$$

For all $i = 1 \dots m$

(where i represents each dimension like Weight, Volume, etc.)

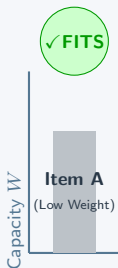
Visual Comparison: Why is MKP Harder?

The "Hidden" Constraint Problem

In Standard Knapsack, we only check one dimension. In MKP, an item might look "good" in one dimension but "break" another.

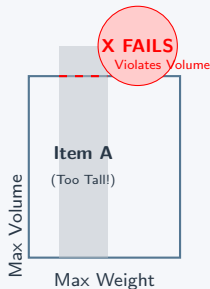
1. Standard Knapsack

(Only Weight matters)



2. Multidimensional

(Weight + Volume)



The Math Unpacked: A System of Constraints

Notation Guide:

Symbol	Name	Meaning in Analogy
n	Items	Total cargo boxes available.
m	Dimensions	The number of "gates" (Weight, Vol, etc.).
x_j	Decision	1 if we load item j , 0 if we leave it.
w_{ij}	Consumption	How much "Weight" item j uses.
C_i	Capacity	The limit of the plane for dimension i .

The "Coupled" Constraints

The difficulty comes because every single item x_j appears in **all** m **equations** simultaneously:

$$\text{Maximize } Z = \sum_{j=1}^n p_j x_j$$

Subject to:

$$\text{Dim 1 (Weight): } w_{1,1}x_1 + w_{1,2}x_2 + \cdots + w_{1,n}x_n \leq C_1$$

$$\text{Dim 2 (Volume): } w_{2,1}x_1 + w_{2,2}x_2 + \cdots + w_{2,n}x_n \leq C_2$$

...

$$\text{Dim } m \text{ (Other): } w_{m,1}x_1 + w_{m,2}x_2 + \cdots + w_{m,n}x_n \leq C_m$$

Formal Problem Statement

Definition

The Multidimensional Knapsack Problem (MKP) is a combinatorial optimization problem where we must select a subset of n items to maximize total profit, such that the total consumption of resources does not exceed the capacity limits for any of the m available dimensions.

Formulation

Given a set of items $N = \{1, \dots, n\}$ and dimensions $M = \{1, \dots, m\}$:

$$\text{Maximize } z = \sum_{j=1}^n p_j x_j$$

$$\text{Subject to } \sum_{j=1}^n w_{ij} x_j \leq C_i, \quad \forall i \in M$$

Where $x_j \in \{0, 1\}$ is the binary decision variable.

Illustrative examples

Illustrative Example

Consider 4 Items and 2 Dimensions (Weight & Volume):

Item	Profit p_j	Weight w_{1j}	Volume w_{2j}
x_1	10	4	3
x_2	6	2	5
x_3	8	3	2
x_4	5	1	4

Capacity Constraints:

$$\sum_{j=1}^4 w_{1j} x_j \leq C_1 = 6 \qquad \sum_{j=1}^4 w_{2j} x_j \leq C_2 = 7$$

Decision variables: $x_j \in \{0, 1\}$

Illustrative Example

Consider 4 Items and 2 Dimensions (Weight & Volume):

First Attempt (Infeasible): $x_1 = 1, x_3 = 1$ Optimal Profit would be $10 + 8 = 18$, but

$$\underbrace{4 + 3}_{w_{11} + w_{13}} = 7 \not\leq 6 \quad (\text{Weight violated})$$

Feasible Solution: $x_3 = 1, x_4 = 1$

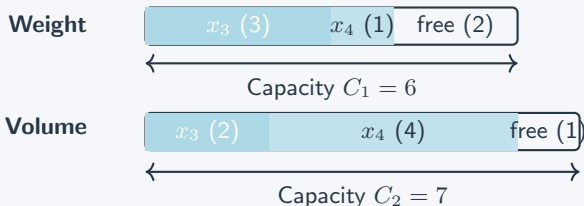
Weight: $3 + 1 = 4 \leq 6$ ✓ Volume: $2 + 4 = 6 \leq 7$ ✓

Optimal Profit = $8 + 5 = 13$

Item	p_j	w_{1j}	w_{2j}	Selected?
x_1	10	4	3	×
x_2	6	2	5	×
x_3	8	3	2	✓
x_4	5	1	4	✓

Optimal Solution: Capacity Visualization

Optimal Profit = 13 (Items x_3 and x_4 selected)

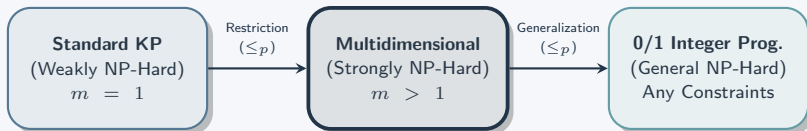


Complexity & Reductions

Complexity Analysis: The "Sandwich" Hierarchy

Where does MKP fit?

To understand the hardness of MKP, we locate it between two well-known problems using **Polynomial-Time Reductions**.



Reduction Direction	Implication for Us
From Knapsack (Left)	Proves MKP is NP-Hard (Inherited difficulty).
To ILP (Right)	Proves MKP is Solvable by general solvers.

Reduction 1: From Standard Knapsack ($KP \leq_p MKP$)

The Logic: "Restriction"

Think of MKP as a "**Universal Tool**" that handles m constraints. Standard Knapsack (KP) is just a specific job where $m = 1$.

"If I build a machine that can lift ANY object (MKP), it can certainly lift a simple box (KP)."

The Mathematical Proof:

1. **Given:** An instance of Standard Knapsack (Items n , Weights w_j , Capacity C).
2. **Construct:** An MKP instance with dimensions $m = 1$.
3. **Map:** Set the MKP capacity $C_1 = C$ and weights $w_{1j} = w_j$.

Conclusion: Inheritance of Hardness

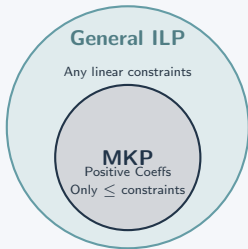
Since Standard Knapsack is known to be **NP-Hard**, and it is just a "special case" embedded inside our problem, **MKP must be at least as hard.**

Reduction 2: Generalization to ILP ($MKP \leq_p ILP$)

What is Integer Linear Programming (ILP)?

ILP is the "Universal Parent" of optimization problems. It can solve **any** problem that has:

- A linear objective function.
- Any set of linear constraints (equalities or inequalities).
- Integer decision variables.



The Argument:

- MKP constraints are just a **specific type** of linear constraint.
- Therefore, $MKP \subset ILP$.
- **Implication:** Any tool that solves the "Parent" (ILP) automatically solves the "Child" (MKP).

The Matrix Transformation

The Translation Step

To use an ILP solver (like CPLEX), we must translate our specific w_{ij} and C_i values into the standard Matrix Format: $\mathbf{Ax} \leq \mathbf{b}$.

Constructing the Matrices:

$$\underbrace{\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,n} \\ w_{2,1} & w_{2,2} & \dots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1} & w_{m,2} & \dots & w_{m,n} \end{bmatrix}}_{\substack{\mathbf{A} \text{ (Constraint Matrix)} \\ \text{Size: } m \times n}} \times \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{\substack{\mathbf{x} \text{ (Decision Vector)} \\ \text{Size: } n \times 1}} \leq \underbrace{\begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_m \end{bmatrix}}_{\substack{\mathbf{b} \text{ (Capacity Vector)} \\ \text{Size: } m \times 1}}$$

Solvability Proof

Since we successfully mapped every MKP variable to a valid ILP matrix component, we have proven that **MKP is solvable** by any standard Branch-and-Cut algorithm used in commercial software.

Existing Algorithms

Comprehensive Algorithm Categories for MKP

Five major categories of solution approaches

- **Exact:** Optimal solutions, but exponential time complexity.
- **Approximation:** Provable bounds within polynomial time.
- **Randomized:** Probabilistic search where quality $\propto$ samples.
- **Heuristic:** Fast, problem-specific, and near-optimal.
- **Metaheuristic:** General-purpose optimization with exploration balance.

Category 1: Exact Exponential Algorithms

Guaranteed Optimal Solutions

Algorithm	Description	Ref
Branch-and-Bound	LP relaxation upper bounds; tree pruning when bound $\leq$ current best	[1]
Dynamic Programming	m -dim DP table; state = remaining capacity vector $\mathbf{r} \in \mathbb{Z}^m$	[2]

Category 2: Approximation Algorithms

Provable Bounds, Polynomial Time

Algorithm	Description	Ref
PTAS	Enumerate size- k subsets; greedily fill remaining capacity; $(1-\varepsilon)$ -ratio for fixed m	[3]
Lagrangian Relaxation	Dualize capacity constraints; tighten multipliers via subgradient iterations	[4]

Category 3: Randomized Algorithms

Probabilistic Exploration

Algorithm		Description	Ref
Randomized (GRASP)	Greedy	Top- k RCL by Toyoda ratio; pick uniformly at random; repeat T times; return best	[5]
Randomized Rounding		Sample $x_j \sim \text{Bern}(x_j^*)$ from LP; repair; repeat for concentration guarantee	[6]

Category 4: Heuristic Algorithms

Fast, Near-Optimal Search

Algorithm	Description	Ref
Toyoda's Method	Primal greedy; dynamic ratio $\mu_j = p_j / \sum_i w_{ij} / R_i$; recomputed every step	[7]
Pivot-and-Complement	Swap selected j_{out} for unselected j_{in} when profitable and feasible	[8]

Category 5: Metaheuristic Algorithms

General-Purpose Optimization

Algorithm	Description	Ref
Tabu Search	1-flip neighbourhood; tabu list tenure τ ; oscillation between feasible/infeasible regions	[9]
Ant Colony (ACO)	Pheromone trails τ_j guide item selection; evaporation + best-ant deposit	[10]

Implemented Algorithms

Toyoda Heuristic

Toyota Heuristic for MKP

Algorithm: Toyota's Gradient Method

1. Initialize solution vector $x \leftarrow \mathbf{0}$ and remaining capacities $R \leftarrow C$
2. Repeat until no more item can be added:
 - Find the unselected feasible item j^* that **maximizes** the ratio:

$$r_j = \frac{p_j}{\sum_{i=1}^m \frac{w_{ij}}{R_i}}$$

(If $R_i = 0$, use $R_i = 1$ to avoid division by zero)

- If no feasible item exists, **break**
 - Select the item: $x_{j^*} \leftarrow 1$
 - Update remaining capacities: $R \leftarrow R - w_{:,j^*}$
3. Compute total profit $= p^\top x$
 4. Check feasibility: all constraints satisfied?

Note: The efficiency ratio is **dynamically recomputed** after every selection using the **current** remaining capacities.

Toyoda's Heuristic – Step-by-Step Simulation

3 Items, 2 Dimensions. Capacities: $C_1 = 6$, $C_2 = 6$.

Item j	w_{1j}	w_{2j}	p_j
1	2	3	4
2	4	2	5
3	3	4	3

Initial State:

$$x = [0, 0, 0], \quad R = [6, 6]$$

Toyota's Heuristic – Iteration 1

Compute efficiency ratios $r_j = \frac{p_j}{\sum_i \frac{w_{ij}}{R_i}}$

$$r_1 \approx 4.80, \quad r_2 = \mathbf{5.00}, \quad r_3 \approx 2.57$$

Item	r_j	Feasible?	Action
1	4.80	✓	–
2	5.00	✓	SELECT $j^* = 2$
3	2.57	✓	–

Update after selecting Item 2:

$$x = [0, 1, 0], \quad R = [2, 4]$$

Toyota's Heuristic – Iteration 2

Remaining capacities: $R = [2, 4]$

$$r_1 \approx 2.29, \quad r_3 = 1.20$$

Feasibility Check:

- Item 1: Feasible (✓)
- Item 3: Infeasible (violates Dimension 1: $3 > 2$)

Item	r_j	Feasible?	Action
1	2.29	✓	SELECT $j^* = 1$
3	1.20	×	Skipped

Update:

$$x = [1, 1, 0], \quad R = [0, 1]$$

Toyota's Heuristic – Iteration 3 & Termination

Remaining capacities: $R = [0, 1]$

Only Item 3 left:

$3 > 0 \Rightarrow$ **Infeasible (Dim 1 violated)**

Item	r_j	Feasible?	Action
3	—	×	Skipped

No more feasible items can be added.

Algorithm terminates.

$$\mathbf{x} = [1, 1, 0]$$

Feasibility Verification:

$$\text{Dimension 1: } 2(1) + 4(1) + 3(0) = 6 \leq C_1 = 6 \quad \checkmark$$

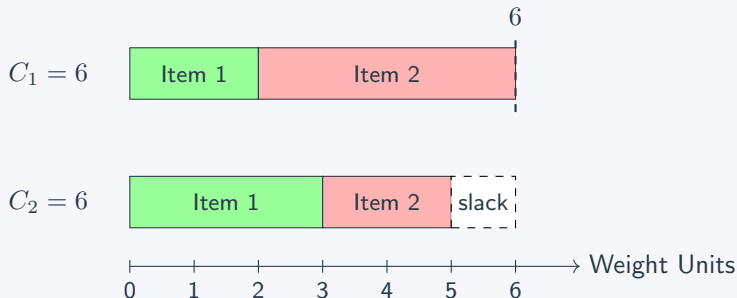
$$\text{Dimension 2: } 3(1) + 2(1) + 4(0) = 5 \leq C_2 = 6 \quad \checkmark$$

Total Profit:

$$z = p^\top x = 4(1) + 5(1) + 3(0) = \mathbf{9}$$

Iteration Summary & Capacity Usage

Iter.	Selected j^*	r_{j^*}	Profit Added	R_1	R_2	Cum. Profit
1	Item 2	5.00	5	2	4	5
2	Item 1	2.29	4	0	1	9
3	— (none feasible)	—	0	0	1	9



Improved Version of Toyoda's Heuristic

Toyota Modification: 1-Swap Local Search

Problem

Pure greedy construction **gets trapped** in a local optimum: once an item is selected it is never reconsidered, even if removing it would allow a more profitable combination to fit.

1-Swap Improvement Phase

After Phase 1, iterate over every **selected** item j_{out} and search for the best **unselected** item j_{in} such that:

- (a) j_{in} is **feasible** after removing j_{out} :

$$w_{i,j_{\text{in}}} \leq r_i + w_{i,j_{\text{out}}} \quad \forall i = 1, \dots, m$$

- (b) The swap yields a **strict profit gain**:

$$p_{j_{\text{in}}} - p_{j_{\text{out}}} > 0$$

Perform the best such swap; repeat for at most `max_swap_rounds` rounds or until no improvement is found.

Toyota Modification: Fill-Up Pass Repositioned

Problem

The original Fill-Up pass runs **immediately** after Phase 1 construction. At that point the remaining capacity is at its tightest, so very few unselected items can actually be inserted — the fill-up provides **little benefit**.

Fill-Up After Swap Phase

Move the Fill-Up pass to **Phase 3**, after the 1-swap local search:

1. Collect all still-unselected items U .
2. Rank them by **weight-sum density**:

$$d_j = \frac{p_j}{\sum_{i=1}^m w_{ij}}, \quad j \in U$$

3. Greedily insert items in decreasing d_j order whenever **feasible** under current remaining capacities r .

Swaps in Phase 2 can **free residual capacity**; placing the Fill-Up afterwards lets it exploit that freed space.

Combined swap + fill-up $\Rightarrow$ **more thorough capacity utilisation**.

Modified Toyota: Algorithm Overview

Three-Phase Structure

1. Phase 1 — Greedy Construction

Iteratively select the item with the highest Toyota ratio (recomputed at each step using current remaining capacities r).

2. Phase 2 — 1-Swap Local Search

For each selected j_{out} , find the best unselected j_{in} that fits and improves profit. Repeat up to `max_swap_rounds` times.

3. Phase 3 — Fill-Up Pass

Insert remaining items by density d_j wherever capacity allows.

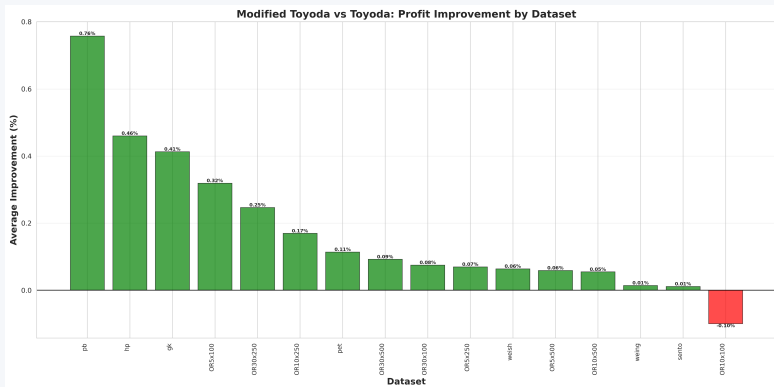
Why Each Phase Helps

Phase	Contribution
1	Builds a high-quality initial solution.
2	Escapes greedy local optima by reconsidering poor selections.
3	Fills residual capacity freed by swaps, squeezing out extra profit.

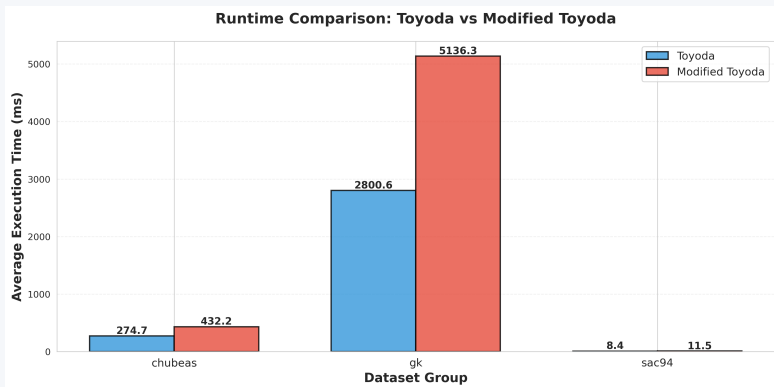
Overall complexity per call:

$$O(n^2m + \text{rounds} \cdot n^2m) = O(n^2m)$$

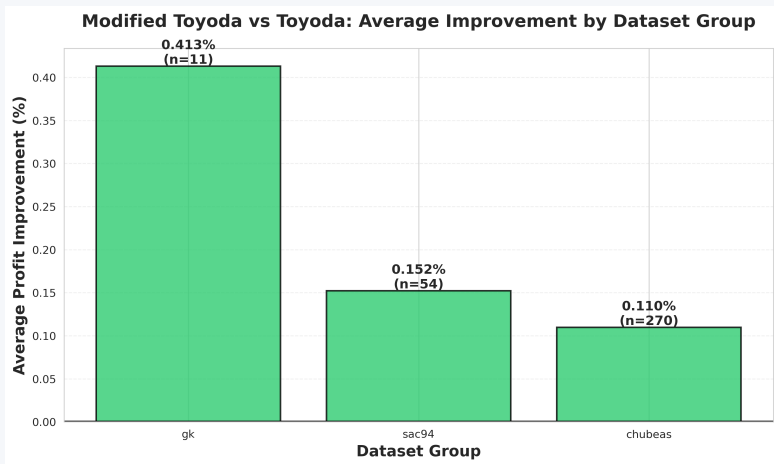
Empirical Results: Toyota vs. Modified Toyota – Improvement by Dataset



Empirical Results: Toyota vs. Modified Toyota – Runtime Comparison



Empirical Results: Toyota vs. Modified Toyota – Runtime Comparison



Randomized Greedy Heuristic

Randomized Greedy Heuristic for MKP

Algorithm: Randomized Greedy – Multi-Restart

1. Initialize best profit $z^* \leftarrow -1$, best solution $x^* \leftarrow \text{None}$, and profit log $\text{all_profits} \leftarrow []$
2. Repeat for T restarts:
 - Run **SingleRun** (construction phase) to obtain solution x
 - Compute profit $z = p^\top x$
 - Append z to all_profits
 - If $z > z^*$, update $z^* \leftarrow z$ and $x^* \leftarrow x$
3. Check feasibility: $W \cdot x^* \leq C$ in all dimensions?
4. Return $x^*, z^*, \text{feasible}, \text{all_profits}$

SingleRun – One Randomized Greedy Pass

1. Initialize $x \leftarrow \mathbf{0}$, remaining capacities $R \leftarrow C$, candidate set $U \leftarrow \{0, \dots, n-1\}$
2. **Repeat while** $U \neq \emptyset$:

- *Step 1 – Compute Toyota ratios:*

$$r_j = \frac{p_j}{\sum_{i=1}^m \frac{w_{ij}}{\tilde{R}_i}}, \quad \tilde{R}_i = \max(R_i, 1)$$

- *Step 2 – Build RCL:* Sort by r_j ; take top- k feasible items ($w_{:,j} \leq R$). If $\text{RCL} = \emptyset$, **break**
- *Step 3 – Random selection:* Pick $j^* \sim \text{Uniform}(\text{RCL})$
- *Step 4 – Update:* $x_{j^*} \leftarrow 1$, $R \leftarrow R - w_{:,j^*}$, $U \leftarrow U \setminus \{j^*\}$

3. **Return** x

Note: RCL adds **controlled randomness** $\rightarrow$ better exploration across restarts.

Complexity Analysis

Scope	Expression	Bottleneck
Per step (ratio recompute)	$O(U \cdot m)$	ratio over remaining items
Per single run	$O(n^2 \cdot m)$	$n \text{ steps} \times O(n \cdot m)$
Total (all restarts)	$O(T \cdot n^2 \cdot m)$	T independent runs

Randomized Greedy – Step-by-Step Simulation

Randomized Greedy – Step-by-Step Simulation

3 Items, 2 Dimensions.

Capacities: $C_1 = 6$, $C_2 = 6$. RCL size: $k = 2$.

Item j	w_{1j}	w_{2j}	p_j
1	2	3	4
2	4	2	5
3	3	4	3

Initial State:

$$x = [0, 0, 0], \quad R = [6, 6], \quad U = \{1, 2, 3\}$$

Run 1 of T (e.g., $T = 3$ shown)

Iteration 1

Step 1 – Toyoda Ratio ($R_1 = 6$, $R_2 = 6$):

$$r_1 = \frac{4}{\frac{2}{6} + \frac{3}{6}} = \frac{4}{0.833} \approx 4.80 \quad r_2 = \frac{5}{\frac{4}{6} + \frac{2}{6}} = 5.00 \quad r_3 = \frac{3}{\frac{3}{6} + \frac{4}{6}} \approx 2.57$$

Step 2 – Build RCL ($k = 2$):

Rank	Item	r_j	Feasible?
1	2	5.00	✓ → add to RCL
2	1	4.80	✓ → add to RCL
3	3	2.57	✓ (RCL full, skip)

$$\text{RCL} = \{2, 1\}$$

Step 3 – Random Selection: $j^* = 1$ (suppose random picks Item 1)

Step 4 – Update:

$$x_1 \leftarrow 1, \quad R = [4, 3], \quad U \leftarrow \{2, 3\}$$

Run 1 of T (continued)

Iteration 2

Step 1 – Toyoda Ratio ($R_1 = 4$, $R_2 = 3$):

$$r_2 \approx 3.00 \quad r_3 \approx 1.44$$

Step 2 – Build RCL:

Rank	Item	r_j	Feasible?
1	2	3.00	✓ → add
2	3	1.44	× (Dim 2) → skip

$$\text{RCL} = \{2\}$$

Step 3 – Random Selection: $j^* = 2$ (only one item in RCL)

Step 4 – Update:

$$x = [1, 1, 0], \quad R = [0, 1], \quad U \leftarrow \{3\}$$

Run 1 of T (continued)

Iteration 3

Step 1 – Feasibility Check for Item 3:

$$w_{13} = 3 > R_1 = 0 \Rightarrow \text{Infeasible.}$$

Step 2 – Build RCL:

Item	r_j	Feasible?	Action
3	—	×	Skipped

$$\text{RCL} = \emptyset \Rightarrow \text{break.}$$

Run 1 Result:

$$x^{(1)} = [1, 1, 0], \quad z^{(1)} = 9$$

Modifications to Randomized Greedy

Randomized Greedy Modification: Adaptive α + Local Search

Problem

Fixed cardinality k is **rigid** – same number of candidates regardless of ratio spread.

- Early iterations need high diversity $\Rightarrow$ high α
- Late iterations need greediness $\Rightarrow$ low α
- No local improvement after construction

Solution: Adaptive Value-Based RCL + 1-Swap Local Search

- **Value-based RCL** :

$$\text{threshold} = r_{\max} - \alpha (r_{\max} - r_{\min})$$

- **Adaptive α** decays over iterations:

$$\alpha_t = \alpha_{\text{start}} + (0.1 - \alpha_{\text{start}}) \cdot \frac{t}{n_{\text{iter}}}$$

- **1-swap local search** applied to every constructed solution

Randomized Greedy Modification: Elite Seeding + Dead Item Pruning

Problem

- Repeated feasibility checks on **all** unselected items
- Early random restarts may accept weak solutions as incumbent

Solution: Elite Seeding + Permanent Dead Item Pruning

- **Elite Seeding:** Run TOYODA once before any random restart

$$(x^*, z^*) \leftarrow \text{TOYODA}(\text{instance})$$

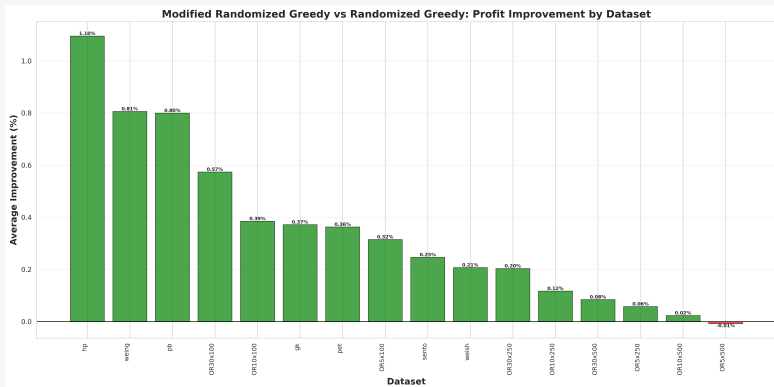
- **Permanent Dead Item Pruning:** After each selection, remove items that can never fit again

$$U \leftarrow \{j \in U \mid w_{:,j} \leq R \ \forall i\}$$

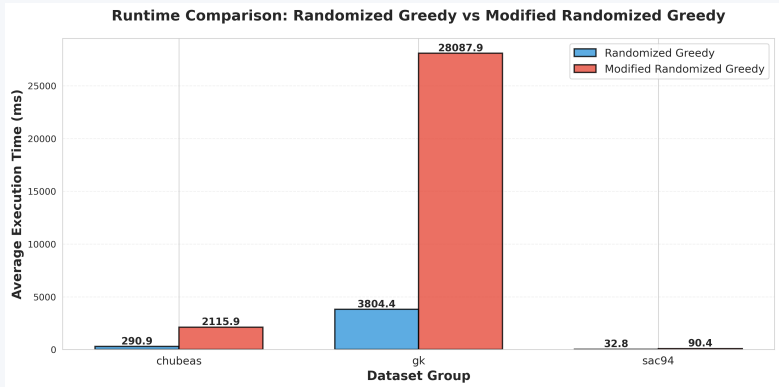
Key Benefits

- Guaranteed $z^{\text{modified}} \geq z^{\text{Toyoda}}$
- Work per step shrinks from $O(n \cdot m)$ to $O(|U| \cdot m)$
- Strong baseline + local search = significantly better final solutions

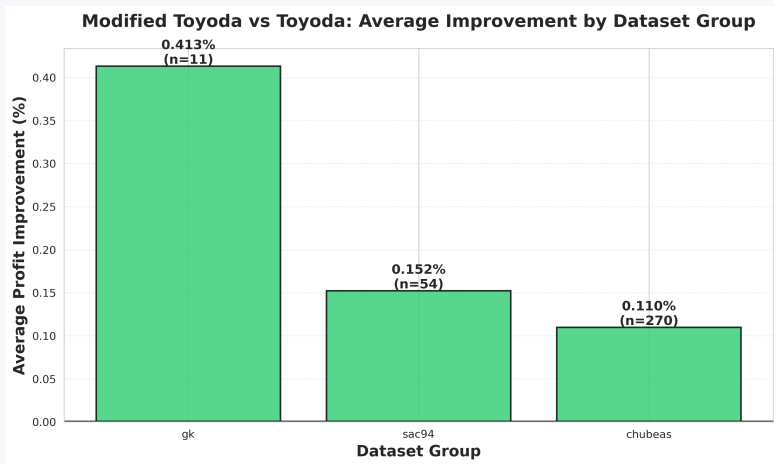
Empirical Results: Randomized Greedy – Improvement by Dataset



Empirical Results: Randomized Greedy – Runtime Comparison



Empirical Results: Randomized Greedy – Avg Improvement by Group



Dataset Details

Dataset

Source: OR-Library (Chu & Beasley, 1998) — 344 benchmark .dat files.

Dataset:

[Benchmark Instances for the Multidimensional Knapsack Problem](#)

Chubeas (Main Benchmark)

Folder	Configuration
OR5x100	$n = 100, m = 5$
OR10x100	$n = 100, m = 10$
OR30x100	$n = 100, m = 30$
OR5x250	$n = 250, m = 5$
OR10x250	$n = 250, m = 10$
OR30x250	$n = 250, m = 30$
OR5x500	$n = 500, m = 5$
OR10x500	$n = 500, m = 10$
OR30x500	$n = 500, m = 30$

$\alpha \in \{0.25, 0.50, 0.75\}$ (30 files per group)

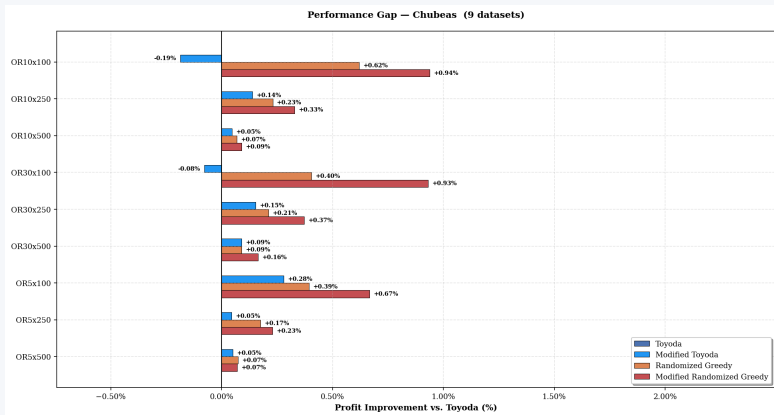
Additional Benchmark Sets

- gk/ Glover–Kochenberger instances
Optimum known for each instance
- sac94/ SAC-94 benchmark set
Includes: weish, sento, weing, etc.

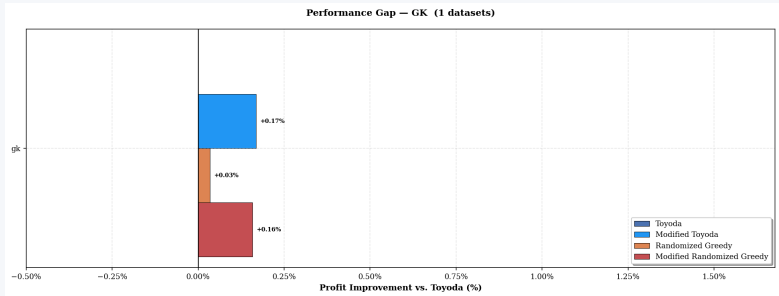
Problem Scale Summary

Parameter	Range
Items (n)	100 $\rightarrow$ 500
Dimensions (m)	5 $\rightarrow$ 30
Total files	344

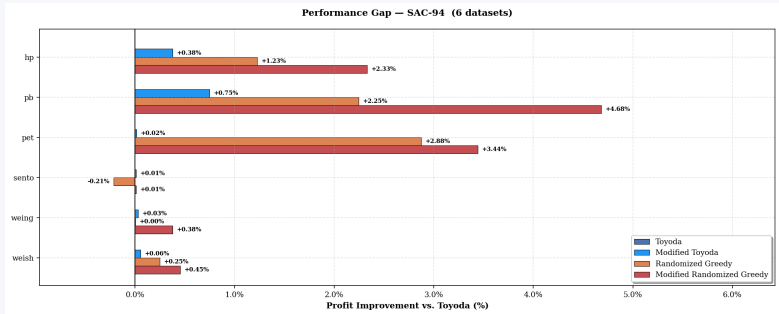
Empirical Results – Comparative to Toyota's Heuristic on Chubeas Dataset



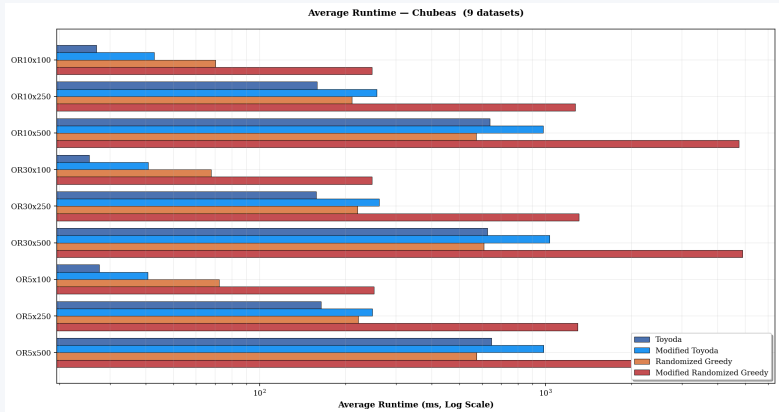
Empirical Results – Comparative to Toyota's Heuristic on gk Dataset



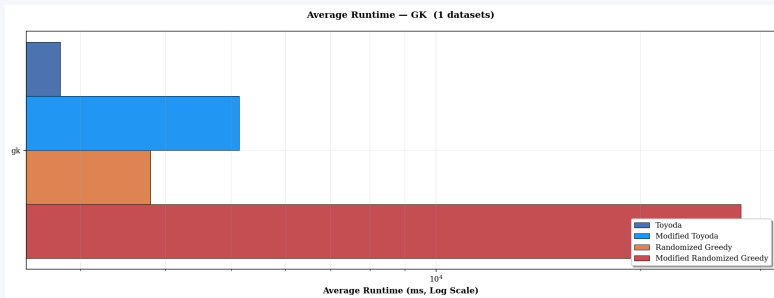
Empirical Results – Comparative to Toyota's Heuristic on sac94 Dataset



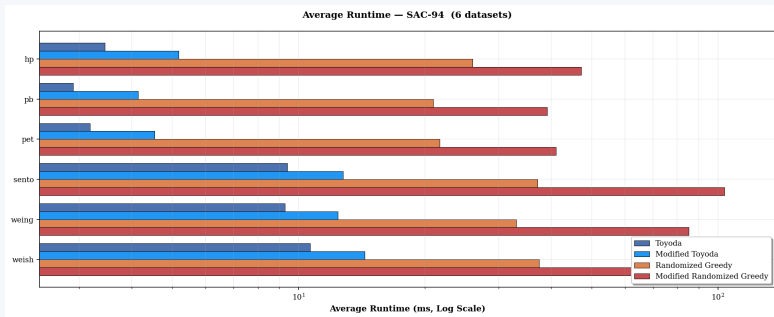
Empirical Results: Runtime Comparison on Chubeas Dataset



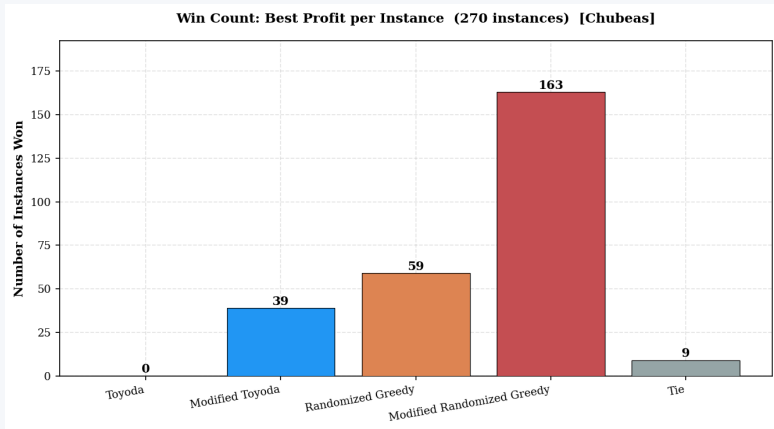
Empirical Results: Runtime Comparison on gk Dataset



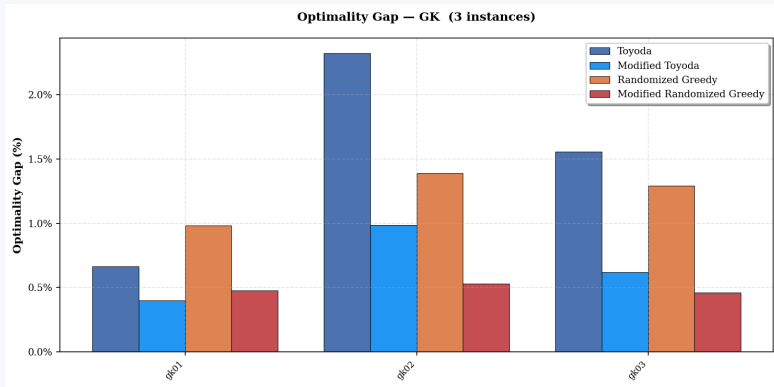
Empirical Results: Runtime Comparison on sac94 Dataset



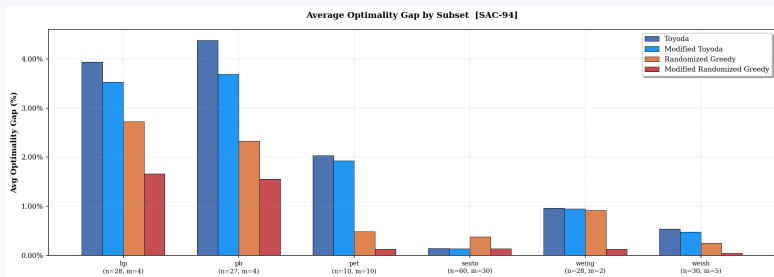
Empirical Results: Win Counts – Chubeas Dataset



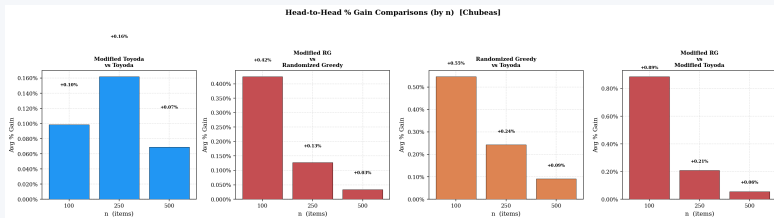
Empirical Results: Optimality Gap – GK Instances



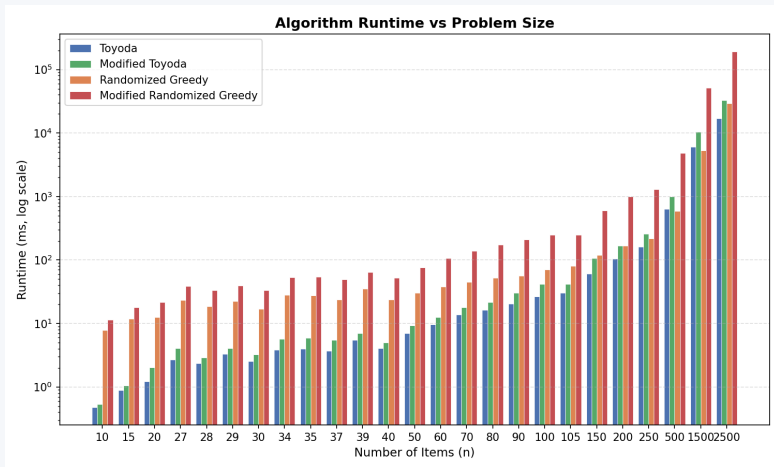
Empirical Results: Optimality Gap – SAC-94 Instances



Empirical Results: Optimality Gap – SAC-94 Instances



Empirical Results: Optimality Gap – SAC-94 Instances



GitHub Repository:

<https://github.com/Tusherbhomik/Algorithm-Sessional-Project--461>

Quick Reproduction Command

```
python run_full_experiment.py
```

Contains all code, datasets, and instructions required to reproduce the plots shown in this presentation.

References

1. Gavish, B., & Pirkul, H. (1985). *Efficient algorithms for solving multiconstraint zero-one knapsack problems to optimality*. Mathematical Programming, 31, 78–105.
researchgate.net/publication/227440053
2. Gilmore, P.C., & Gomory, R.E. (1966). *The theory and computation of knapsack functions*. Operations Research, 14(6), 1045–1074.
researchgate.net/publication/220669373
3. Frieze, A.M., & Clarke, M.R.B. (1984). *Approximation algorithms for the m-dimensional 0-1 knapsack problem*. European J. Oper. Res., 15(1), 100–109.
researchgate.net/publication/222351938
4. Lemaréchal, C. (2001). *Lagrangian Relaxation*. In Computational Combinatorial Optimization, Springer.
researchgate.net/publication/220669373
5. Feo, T.A., & Resende, M.G.C. (1995). *Greedy Randomized Adaptive Search Procedures*. J. Global Optimization, 6, 109–133.
mauricio.resende.info/doc/gtut.pdf
6. Raghavan, P., & Thompson, C.D. (1987). *Randomized rounding*. Combinatorica, 7(4), 365–374.
researchgate.net/publication/220694474
7. Toyoda, Y. (1975). *A simplified algorithm for obtaining approximate solutions to zero-one programming problems*. Management Science, 21(12), 1417–1427.
[stars.library.ucf.edu \(Akin 2009, Ch.2\)](https://stars.library.ucf.edu/Akin2009/Ch.2)
8. Glover, F., & Woolsey, E. (1974). *Converting the 0-1 polynomial programming problem to a 0-1 linear program*. Operations Research, 22(1), 180–182.
researchgate.net/publication/238875249
9. Hanafi, S., & Freille, A. (1998). *An efficient tabu search approach for the 0-1 multidimensional knapsack problem*. European J. Oper. Res., 106(2-3), 659–675.
researchgate.net/publication/2396195
10. Leguizamón, G., & Michalewicz, Z. (1999). *A new version of ant system for subset problems*. Proc. IEEE CEC.
researchgate.net/publication/255645639

Thank You!